



PYTHON - SE - BACKEND

SOFTWARE ENGINEERING – MODULE 3

Introduction to OOPS Programming

SESSION / ASSIGNMENT – 2

TASKS :

1. Create a class called Playlist with properties: name (string), createdOn (date), and isPublic (boolean). Instantiate an object of Playlist and print all its properties.



```
#include <iostream>
#include <string>
using namespace std;

class Playlist
{
public:
    string name;
    string createdOn;
    bool isPublic;

    Playlist(string n, string date, bool publicStatus)
    {
        name = n;
        createdOn = date;
        isPublic = publicStatus;
    }
};

int main()
{
    Playlist playlist("My Favorites", "18-08-2026", true);

    cout << "Name: " << playlist.name << endl;
    cout << "Created On: " << playlist.createdOn << endl;
    cout << "Public: " << playlist.isPublic << endl;

    return 0;
}
```

OUTPUT:

```
Name: My Favorites  
Created On: 18-08-2026  
Public: 1
```

2. Add a member function to your Playlist class named togglePublic() that switches isPublic between true and false each time it is called. Demonstrate by toggling the value twice and printing the result each time.



```
#include <iostream>
#include <string>
using namespace std;

class Playlist
{
public:
    string name;
    string createdOn;
    bool isPublic;

    Playlist(string n, string date, bool publicStatus)
    {
        name = n;
        createdOn = date;
        isPublic = publicStatus;
    }

    void togglePublic()
    {
        isPublic = !isPublic;
    }
};

int main()
{
    Playlist playlist("My Favorites", "18-08-2026", true);

    cout << "Initial: " << playlist.isPublic << endl;

    playlist.togglePublic();
    cout << "After first toggle: " << playlist.isPublic << endl;

    playlist.togglePublic();
    cout << "After second toggle: " << playlist.isPublic << endl;

    return 0;
}
```

```
}
```

OUTPUT:

```
Initial: 1  
After first toggle: 0  
After second toggle: 1
```

3. Create a class called **FoodOrder** with properties: **orderId** (number), **restaurantName** (string), and **isDelivered** (boolean). Write a member function **markDelivered()** that sets **isDelivered** to true and prints a message. Instantiate **FoodOrder** and call **markDelivered()**.



```
#include <iostream>
#include <string>
using namespace std;

class FoodOrder
{
public:
    int orderId;
    string restaurantName;
    bool isDelivered;

    FoodOrder(int id, string restaurant, bool delivered)
    {
        orderId = id;
        restaurantName = restaurant;
        isDelivered = delivered;
    }

    void markDelivered()
    {
        isDelivered = true;
        cout << "Order " << orderId << " has been delivered." << endl;
    }
};

int main()
{
    FoodOrder order(101, "Pizza Hut", false);

    order.markDelivered();

    cout << "Delivered: " << order.isDelivered << endl;

    return 0;
}
```

OUTPUT:

```
Order 101 has been delivered.  
Delivered: 1
```

4. Extend your Playlist class to include a member function addSong(songTitle) that adds the song title to an array property called songs. Demonstrate by adding three song titles and displaying the updated songs list.

Hint: Initialize songs as an empty array inside the constructor.



```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Playlist
{
public:
    string name;
    string createdOn;
    bool isPublic;
    vector<string> songs;

    Playlist(string n, string date, bool publicStatus)
    {
        name = n;
        createdOn = date;
        isPublic = publicStatus;
    }

    void addSong(string songTitle)
    {
        songs.push_back(songTitle);
    }

    void displaySongs()
    {
        for(int i = 0; i < songs.size(); i++)
        {
            cout << songs[i] << endl;
        }
    }
};

int main()
```

```

{
    Playlist playlist("My Favorites", "18-08-2026", true);

    playlist.addSong("Tum Hi Ho");
    playlist.addSong("Believer");
    playlist.addSong("Perfect");

    cout << "Songs in Playlist:" << endl;

    playlist.displaySongs();

    return 0;
}

```

OUTPUT:

```

Songs in Playlist:
Tum Hi Ho
Believer
Perfect

```

5. Refactor your FoodOrder class so that the constructor takes an object as a parameter (with keys orderId, restaurantName, isDelivered) instead of separate arguments. Update your instantiation code to use this new constructor signature.



```

#include <iostream>
#include <string>
using namespace std;

struct OrderDetails
{
    int orderId;
    string restaurantName;
    bool isDelivered;
};

class FoodOrder
{
public:
    int orderId;
    string restaurantName;
    bool isDelivered;

    FoodOrder(OrderDetails order)
    {
        orderId = order.orderId;
        restaurantName = order.restaurantName;
    }
}

```

```

        isDelivered = order.isDelivered;
    }

    void markDelivered()
    {
        isDelivered = true;
        cout << "Order " << orderId << " has been delivered." << endl;
    }
};

int main()
{
    OrderDetails details = {101, "Pizza Hut", false};

    FoodOrder order(details);

    cout << "Order ID: " << order.orderId << endl;
    cout << "Restaurant: " << order.restaurantName << endl;
    cout << "Delivered: " << order.isDelivered << endl;

    order.markDelivered();

    return 0;
}

```

OUTPUT:

```

Order ID: 101
Restaurant: Pizza Hut
Delivered: 0
Order 101 has been delivered.

```